

Data Preprocessing

Data preprocessing adalah proses persiapan data sebelum dilakukan analisis atau pemodelan. Ini melibatkan serangkaian langkah seperti pembersihan data (cleaning), penghapusan nilai yang hilang, penanganan outlier, normalisasi, transformasi, atau pengkodean ulang, serta pemilihan fitur atau variabel yang relevan untuk analisis atau model tertentu. Tujuan dari data preprocessing adalah untuk meningkatkan kualitas dan keterampilan data sehingga memungkinkan untuk mendapatkan hasil analisis yang lebih akurat dan relevan.

read the dataset

silahkan download dataset di <https://www.kaggle.com/datasets/yasserh/titanic-dataset>

Download dan gunakan google drive and github anda sebagai media penyimpanan dataset

```
In [ ]: import pandas as pd
df = pd.read_csv('https://raw.githubusercontent.com/ganjar87/data_science_practice/main/train.csv', delimiter=',')
df.head()
```

Out[]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

```
In [ ]: df.tail()
```

Out[]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	211536	13.00	NaN	S
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112053	30.00	B42	S
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	W./C. 6607	23.45	NaN	S
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369	30.00	C148	C
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	370376	7.75	NaN	Q

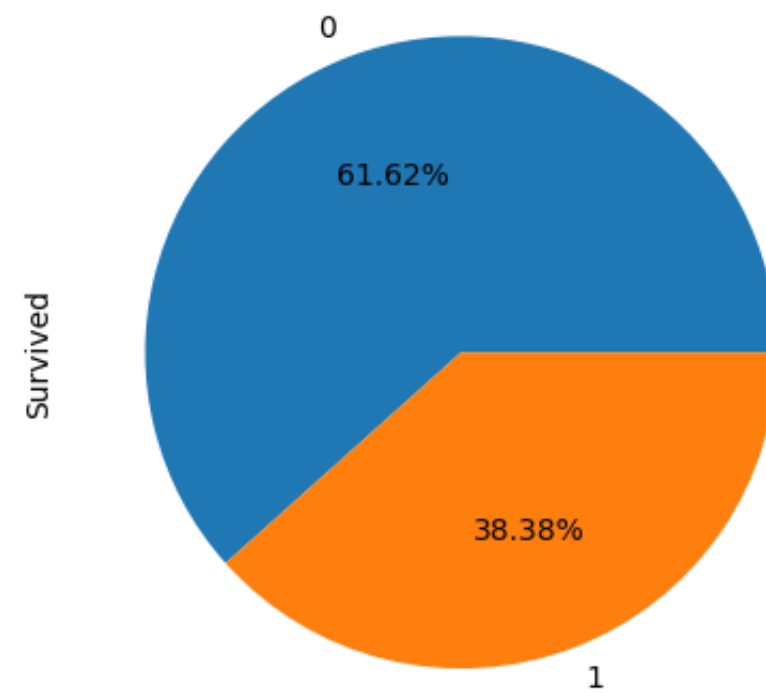
```
In [ ]: # Melihat bentuk dataset. Baris, kolom
df.shape
```

Out[]: (891, 12)

check percentage of target class

```
In [ ]: import matplotlib.pyplot as plt

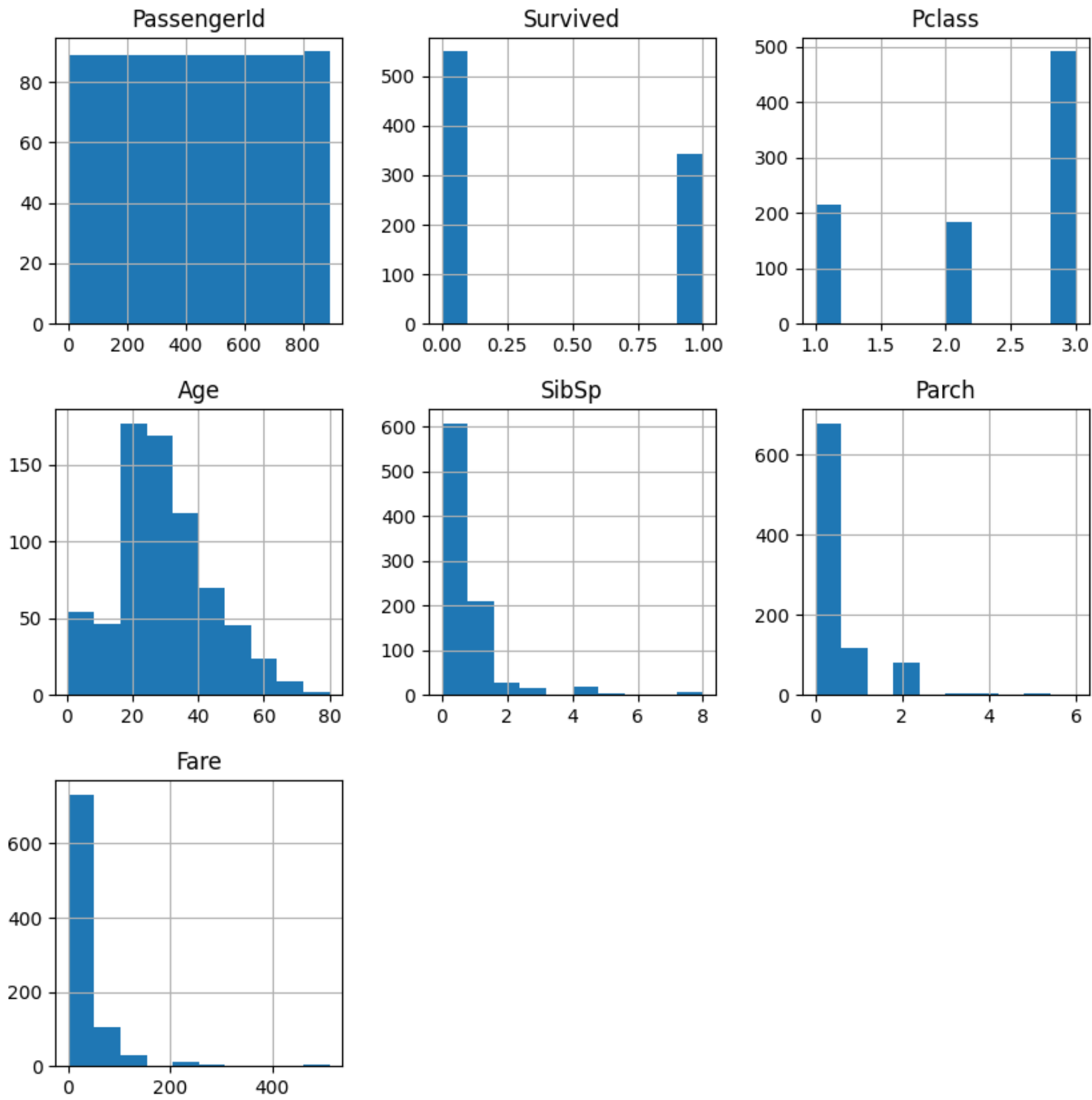
data = df['Survived'].value_counts()
data.plot(kind='pie', autopct='%0.2f%%')
plt.show()
```



melihat histogram

```
In [ ]: df.hist(figsize=(10,10))
plt.show()

# menggunakan histogram, untuk melihat distribusi nya
```



melihat korelasi antar variabel

Korelasi antar variabel adalah ukuran statistik yang mengukur seberapa erat hubungan atau hubungan linear antara dua variabel dalam sebuah dataset, menunjukkan arah dan kekuatan hubungan antara keduanya. Semakin dekat nilai korelasi dengan 1 atau -1, semakin kuat hubungannya; nilai positif menunjukkan korelasi positif (ketika satu variabel naik, yang lain cenderung naik juga), sedangkan nilai negatif menunjukkan korelasi negatif (ketika satu variabel naik, yang lain cenderung turun). Sebaliknya, nilai korelasi yang mendekati 0 menunjukkan bahwa tidak ada hubungan linier yang signifikan antara variabel tersebut.

```
In [ ]: df.corr(numeric_only = True)
```

Out []:

	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare
PassengerId	1.000000	-0.005007	-0.035144	0.036847	-0.057527	-0.001652	0.012658
Survived	-0.005007	1.000000	-0.338481	-0.077221	-0.035322	0.081629	0.257307
Pclass	-0.035144	-0.338481	1.000000	-0.369226	0.083081	0.018443	-0.549500
Age	0.036847	-0.077221	-0.369226	1.000000	-0.308247	-0.189119	0.096067
SibSp	-0.057527	-0.035322	0.083081	-0.308247	1.000000	0.414838	0.159651
Parch	-0.001652	0.081629	0.018443	-0.189119	0.414838	1.000000	0.216225
Fare	0.012658	0.257307	-0.549500	0.096067	0.159651	0.216225	1.000000

melihat descriptive statistic

Descriptive statistics adalah metode statistik yang digunakan untuk meringkas, menggambarkan, dan memahami distribusi dan karakteristik utama dari suatu dataset, termasuk mean, median, modus, kuartil, varians, dan deviasi standar. Ini membantu dalam memberikan gambaran komprehensif tentang data tanpa membuat kesimpulan atau generalisasi yang lebih lanjut.

```
In [ ]: df.describe()
```

Out []:

	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare
count	891.000000	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000
mean	446.000000	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
std	257.353842	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
min	1.000000	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	223.500000	0.000000	2.000000	20.125000	0.000000	0.000000	7.910400
50%	446.000000	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%	668.500000	1.000000	3.000000	38.000000	1.000000	0.000000	31.000000
max	891.000000	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

check missing values

Missing values, atau nilai yang hilang, merujuk pada keadaan di mana tidak ada data yang tersedia atau tercatat untuk beberapa observasi dalam sebuah dataset. Hal ini bisa terjadi karena berbagai alasan, termasuk kesalahan pengumpulan data, kerusakan perangkat, atau keengganan responden. Penanganan yang tepat terhadap nilai yang hilang penting dalam analisis data, karena bisa mempengaruhi hasil analisis atau model yang dibangun.

```
In [ ]: df.isnull().sum()
```

```
Out[ ]: PassengerId      0
Survived      0
Pclass        0
Name          0
Sex           0
Age          177
SibSp         0
Parch         0
Ticket        0
Fare          0
Cabin        687
Embarked      2
dtype: int64
```

```
In [ ]: # Missing Value Check
df.isna().sum()
```

```
Out[ ]: PassengerId      0
Survived      0
Pclass        0
Name          0
Sex           0
Age          177
SibSp         0
Parch         0
Ticket        0
Fare          0
Cabin        687
Embarked      2
dtype: int64
```

duplicates

Duplicates, atau data duplikat, merujuk pada keadaan di mana ada observasi atau entri dalam dataset yang memiliki nilai yang sama untuk semua variabel yang relevan. Dalam konteks analisis data, keberadaan data duplikat bisa menjadi masalah karena dapat mempengaruhi hasil analisis dengan cara yang tidak diinginkan. Oleh karena itu, penting untuk mengidentifikasi dan menangani data duplikat dengan tepat, misalnya dengan menghapusnya atau menggabungkannya, tergantung pada kebutuhan analisis atau pemodelan.

```
In [ ]: # Duplicate check
df.duplicated().sum()
```

Out[]: 0

```
In [ ]: df.drop_duplicates(inplace=True)
```

imputation

Imputation adalah proses menggantikan atau memperkirakan nilai yang hilang dalam sebuah dataset dengan nilai yang didasarkan pada pola atau karakteristik lain dari data yang ada. Ini digunakan ketika ada nilai yang hilang dalam dataset, dan imputasi dilakukan untuk menjaga integritas dan keakuratan data sebelum analisis lebih lanjut atau pemodelan.

```
In [ ]: #imputation. kita isi nilai kosong
# kolom numerik
df['Age'].fillna(df['Age'].median(), inplace=True)
#df['Age'].fillna(df['Age'].mean(), inplace=True)

# kolom kategori
df['Cabin'].fillna(df['Cabin'].value_counts().index[0], inplace=True)
df['Embarked'].fillna(df['Embarked'].value_counts().index[0], inplace=True)
df.isnull().sum()
```

Out[]: PassengerId 0
Survived 0
Pclass 0
Name 0
Sex 0
Age 0
SibSp 0
Parch 0
Ticket 0
Fare 0
Cabin 0
Embarked 0
dtype: int64

```
In [ ]: df['Embarked'].value_counts().index[0]
```

Out[]: 'S'

```
In [ ]: df
```


Out []:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	B96 B98	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	B96 B98	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	B96 B98	S
...	...	...	...	...	...	...	...	...	...	...	...	...
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	211536	13.0000	B96 B98	S
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112053	30.0000	B42	S
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	28.0	1	2	W./C. 6607	23.4500	B96 B98	S
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369	30.0000	C148	C
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	370376	7.7500	B96 B98	Q

891 rows × 12 columns

check categorical attributes

In []:

```
# get X and y
df_X = df.drop(['PassengerId', 'Name', 'Survived', 'Cabin', 'Ticket'], axis=1)
df_y = df['Survived']

#check categorical attributes
cats = df_X.select_dtypes(include=['object', 'bool']).columns
print(cats)

Index(['Sex', 'Embarked'], dtype='object')
```

In []: df_X

Out[]:

	Pclass	Sex	Age	SibSp	Parch	Fare	Embarked
0	3	male	22.0	1	0	7.2500	S
1	1	female	38.0	1	0	71.2833	C
2	3	female	26.0	0	0	7.9250	S
3	1	female	35.0	1	0	53.1000	S
4	3	male	35.0	0	0	8.0500	S
...	...	...	...	...	...	...	...
886	2	male	27.0	0	0	13.0000	S
887	1	female	19.0	0	0	30.0000	S
888	3	female	28.0	1	2	23.4500	S
889	1	male	26.0	0	0	30.0000	C
890	3	male	32.0	0	0	7.7500	Q

891 rows × 7 columns

one-hot encoding untuk input features

Style 1

In []:

```
# One Hot Encode
df_onehot = pd.get_dummies(df_X, columns=['Sex', 'Embarked'], drop_first=True)
df_onehot.head()
```

Out[]:

	Pclass	Age	SibSp	Parch	Fare	Sex_male	Embarked_Q	Embarked_S
0	3	22.0	1	0	7.2500	1	0	1
1	1	38.0	1	0	71.2833	0	0	0
2	3	26.0	0	0	7.9250	0	0	1
3	1	35.0	1	0	53.1000	0	0	1
4	3	35.0	0	0	8.0500	1	0	1

Style 2


```
In [ ]: from sklearn.preprocessing import OneHotEncoder
# ohe = OneHotEncoder(sparse_output=False, drop='first')
ohe = OneHotEncoder(sparse_output=False)

ohe.fit(df_X[['Sex', 'Embarked']])

# for col in column_categorical:
df_ohe = ohe.transform(df_X[['Sex', 'Embarked']])

# ohe.categories_
df_ohe
```

```
Out[ ]: array([[0., 1., 0., 0., 1.],
               [1., 0., 1., 0., 0.],
               [1., 0., 0., 0., 1.],
               ...,
               [1., 0., 0., 0., 1.],
               [0., 1., 1., 0., 0.],
               [0., 1., 0., 1., 0.]])
```

label encoding untuk input features

```
In [ ]: #categorical encoding
from sklearn.preprocessing import LabelEncoder
cats = df_X.select_dtypes(include=['object', 'bool']).columns
cat_features = list(cats.values)
cat_en = LabelEncoder()
for i in cat_features:
    df_X[i] = cat_en.fit_transform(df_X[i])

df_X
```

Out []:

	Pclass	Sex	Age	SibSp	Parch	Fare	Embarked
0	3	1	22.0	1	0	7.2500	2
1	1	0	38.0	1	0	71.2833	0
2	3	0	26.0	0	0	7.9250	2
3	1	0	35.0	1	0	53.1000	2
4	3	1	35.0	0	0	8.0500	2
...	...	...	...	...	...	...	...
886	2	1	27.0	0	0	13.0000	2
887	1	0	19.0	0	0	30.0000	2
888	3	0	28.0	1	2	23.4500	2
889	1	1	26.0	0	0	30.0000	0
890	3	1	32.0	0	0	7.7500	1

891 rows x 7 columns

label encoding untuk target class

In []:

```
#label encoding for y
le = LabelEncoder()
le.fit(df_y)
df_y= le.fit_transform(df_y)
df_y
```

[illegible]

correlation coefficient

```
In [ ]: df_y_new = pd.DataFrame(df_y, columns=['Survived'])
df_gabung = pd.concat([df_X, df_y_new], axis=1)
df_gabung.corr()
```

Out []:

	Pclass	Sex	Age	SibSp	Parch	Fare	Embarked	Survived
Pclass	1.000000	0.131900	-0.339898	0.083081	0.018443	-0.549500	0.162098	-0.338481
Sex	0.131900	1.000000	0.081163	-0.114631	-0.245489	-0.182333	0.108262	-0.543351
Age	-0.339898	0.081163	1.000000	-0.233296	-0.172482	0.096688	-0.018754	-0.064910
SibSp	0.083081	-0.114631	-0.233296	1.000000	0.414838	0.159651	0.068230	-0.035322
Parch	0.018443	-0.245489	-0.172482	0.414838	1.000000	0.216225	0.039798	0.081629
Fare	-0.549500	-0.182333	0.096688	0.159651	0.216225	1.000000	-0.224719	0.257307
Embarked	0.162098	0.108262	-0.018754	0.068230	0.039798	-0.224719	1.000000	-0.167675
Survived	-0.338481	-0.543351	-0.064910	-0.035322	0.081629	0.257307	-0.167675	1.000000

train-test split

Train-test split adalah proses membagi dataset menjadi dua subset terpisah: satu untuk melatih model (train set) dan yang lainnya untuk menguji model (test set). Data train digunakan untuk melatih model, sedangkan data test digunakan untuk menguji kinerja model yang dilatih. Tujuannya adalah untuk mengevaluasi seberapa baik model yang dilatih dapat digeneralisasi ke data baru yang belum pernah dilihat sebelumnya.

In []:

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import OneHotEncoder
from sklearn.preprocessing import OrdinalEncoder
from sklearn.preprocessing import MinMaxScaler

#hold out, dibagi menjadi training dan testing set
X_train, X_test, y_train, y_test = train_test_split(df_X, df_y, test_size=0.3, random_state=42)
```

In []:

```
X_train
```

Out []:

	Pclass	Sex	Age	SibSp	Parch	Fare	Embarked
445	1	1	4.0	0	2	81.8583	2
650	3	1	28.0	0	0	7.8958	2
172	3	0	1.0	1	1	11.1333	2
450	2	1	36.0	1	2	27.7500	2
314	2	1	43.0	1	1	26.2500	2
...	...	...	...	...	...	...	...
106	3	0	21.0	0	0	7.6500	2
270	1	1	28.0	0	0	31.0000	2
860	3	1	41.0	2	0	14.1083	2
435	1	0	14.0	1	2	120.0000	2
102	1	1	21.0	0	1	77.2875	2

623 rows × 7 columns

standardization

Standardization adalah proses dalam analisis data yang mengubah distribusi nilai-nilai dari suatu variabel sehingga memiliki mean (rata-rata) nol dan deviasi standar satu. Hal ini dilakukan untuk memastikan bahwa semua variabel memiliki skala yang serupa, sehingga memfasilitasi perbandingan atau pemodelan yang lebih akurat.

In []:

```
from sklearn.preprocessing import StandardScaler
#scaling
scaler = StandardScaler()
scaler.fit(X_train)
X_train = scaler.transform(X_train)
X_test = scaler.transform(X_test)
```

In []:

```
X_train
```

Out []:

```
array([[ -1.63788124,  0.72077194, -1.91971935, ...,  1.99885349,
         0.98099823,  0.57000481],
 [ 0.80326712,  0.72077194, -0.0772525 , ..., -0.47932706,
        -0.46963364,  0.57000481],
 [ 0.80326712, -1.38740139, -2.15002771, ...,  0.75976322,
        -0.40613632,  0.57000481],
 ...,
 [ 0.80326712,  0.72077194,  0.92075038, ..., -0.47932706,
        -0.34778742,  0.57000481],
 [-1.63788124, -1.38740139, -1.15202483, ...,  1.99885349,
         1.72907416,  0.57000481],
 [-1.63788124,  0.72077194, -0.61463866, ...,  0.75976322,
         0.8913508 ,  0.57000481]])
```

normalization

Normalization adalah proses dalam analisis data yang mengubah nilai-nilai dari suatu variabel sehingga rentang nilainya menjadi antara 0 dan 1. Tujuan utamanya adalah untuk menghilangkan perbedaan skala antar variabel, sehingga memungkinkan untuk membandingkan atau memodelkan variabel dengan distribusi yang berbeda secara lebih adil.

```
In [ ]: from sklearn.preprocessing import MinMaxScaler
#scaling
scaler = MinMaxScaler()
scaler.fit(X_train)
X_train = scaler.transform(X_train)
X_test = scaler.transform(X_test)
```

```
In [ ]: X_train
```

```
Out[ ]: array([[0.      , 1.      , 0.04498618, ..., 0.33333333, 0.15977676,
               1.      ],
               [1.      , 1.      , 0.34656949, ..., 0.      , 0.01541158,
               1.      ],
               [1.      , 0.      , 0.00728826, ..., 0.16666667, 0.02173075,
               1.      ],
               ...,
               [1.      , 1.      , 0.50992712, ..., 0.      , 0.02753757,
               1.      ],
               [0.      , 0.      , 0.17064589, ..., 0.33333333, 0.2342244 ,
               1.      ],
               [0.      , 1.      , 0.25860769, ..., 0.16666667, 0.15085515,
               1.      ]])
```

Tugas 2

Dengan menggunakan dataset dibawah ini <https://www.kaggle.com/datasets/amitabhajoy/bengaluru-house-price-data>

Silahkan

1. Membaca dataset menggunakan dataframe
2. Membuat histogram, dan jelaskan
3. Korelasi antar variabel dan jelaskan
4. Membuat descriptive analysis dan jelaskan
5. Cek apakah ada missing value, jika ada apa yg perlu dilakukan.
6. Cek apakah ada duplikasi, jika ada apa yang perlu dilakukan.
7. Lakukan encoding untuk categorical variabel, menggunakan one hot dan label encoding.
8. Pisah dataset menjadi train test (80:20)
9. Lakukan standardization atu normalization